

DOMNIWALLET

Payout Service - Especificação Técnica

Sistema de Payout Multi-Rede: BTC / Liquid / Lightning Network

Documento	Especificação Técnica do Payout Service
Versão	1.0.0
Status	Arquitetura Aprovada - Pronto para Implementação
Data	Janeiro 2026
Público-Alvo	Desenvolvedores, Arquitetos, Operações
Redes Suportadas	Bitcoin, Liquid Network, Lightning Network (LND)
Backend	Go + PostgreSQL + Redis

Índice

1. Sumário Executivo
 - 1.1 Modelo de Operação (Treasury Payout)
 - 1.2 Product Contract
 - 1.3 Decisões Finais
2. Arquitetura do Sistema
 - 2.1 Visão Geral dos Componentes
 - 2.2 Interface dos Executors
 - 2.3 Máquina de Estados
3. Policy Table v1
 - 3.1 Lightning Network
 - 3.2 Bitcoin On-Chain
 - 3.3 Liquid Network
 - 3.4 Treasury Wallet
4. Thresholds em BRL
 - 4.1 Faixas de Valor
 - 4.2 SLOs por Rede
 - 4.3 Limites Operacionais
5. Especificação de Validações
 - 5.1 Códigos de Erro
 - 5.2 Validações por Rede
 - 5.3 Regras de Negócio
6. Fluxos de Execução
 - 6.1 Bitcoin Flow
 - 6.2 Lightning Flow
7. Resiliência
 - 7.1 Circuit Breaker
 - 7.2 Retry Policy
8. Auditoria e Observabilidade
 - 8.1 Schema de Auditoria
 - 8.2 Métricas e Alertas
9. Runbook Operacional
 - 9.1 RBF/CPFP
 - 9.2 DLQ
 - 9.3 Consolidação de UTXOs
10. Checklist de Production Readiness
11. Configuração de Referência

1. Sumário Executivo

DomniWallet Payout Service é um sistema de **Treasury Payout** multi-rede que processa pagamentos em Bitcoin (BTC), Liquid Network (L-BTC) e Lightning Network (LN). O sistema opera a partir de carteiras da tesouraria da empresa, pagando para endereços e invoices controlados pelos clientes.

1.1 Modelo de Operação (Treasury Payout)

Modelo Não-Custodial: O cliente controla sua própria wallet e fornece endereços/invoices para recebimento. A DomniWallet **não custodia fundos de clientes** — apenas opera a tesouraria interna para executar payouts quando Depix é confirmado.

Aspecto	Descrição
Custódia de Clientes	NÃO — cliente mantém suas próprias chaves/wallet
Tesouraria	Carteiras operadas pela empresa para payouts
Fluxo	Depix confirmado → Payout para endereço/invoice do cliente
Risco	Empresa assume risco de liquidez; cliente assume risco de sua wallet

Tabela 1.1: Modelo de Operação

Gatilho de Payout (Depix)

Decisão operacional (v1): DominiPay monitora confirmações do Depix internamente e chama `POST /v1/payouts` somente após `depix_conf=2`. Webhook do gateway Liquid pode ser implementado em v2 para push-based.

Fluxo atual: PIX recebido → Depix enviado → DominiPay aguarda 2 conf → `POST /v1/payouts` → Payout Service executa.

1.2 Product Contract

Definição clara de como destinos são gerenciados por rede:

Rede	Tipo de Destino	Requisito
BTC	Address Book	Endereço fixo cadastrado por usuário (Bech32 preferido)
Liquid	Address Book	Endereço fixo cadastrado por usuário (Confidential opcional)
Lightning	Invoice por Payout	BOLT11 invoice fornecida a cada payout; expiração mínima 10 min

Tabela 1.2: Product Contract por Rede

Lightning não tem endereço fixo: O cliente deve fornecer uma invoice BOLT11 válida para cada payout. LNURL-withdraw pode ser implementado em versão futura como melhoria de UX.

Contrato de Integração DominiPay → Payout Service (MVP)

#	Regra	Detalhe
1	BTC/Liquid: <code>to_address</code> fixo	DominiPay envia endereço do address book do cliente. Validação: <code>validateaddress</code>
2	LN: <code>ln_invoice</code> por payout	BOLT11 obrigatório. Se expirar → <code>PAYOUT_INVOICE_EXPIRED</code> → pedir nova invoice
3	Gatilho: <code>depix_confirmed</code>	DominiPay só chama <code>POST /v1/payouts</code> após Depix com 2 confirmações

Tabela 1.3: Contrato de Integração MVP

1.3 Decisões Finais Aprovadas

Decisão	Resultado	Justificativa
Fallback entre redes	❌ Não automático	Altera expectativa do usuário; fallback via opt-in manual
RBF para BTC	✅ ON por padrão	Resolve stuck tx com baixa complexidade via <code>bumpfee</code>
CPFP para BTC	⚠️ Opcional	Somente se controlamos o output de change
Zero-amount invoices	✅ Aceitar com condição	Somente se payout tem valor definido; enviar via <code>amt_msat</code>
MPP (Multi-Path)	✅ ON acima de 100k sats	Aumenta taxa de sucesso em valores maiores
Webhook Depix	🔍 Verificar	Confirmar se assinatura implementada

Tabela 1.1: Decisões Finais Aprovadas

1.2 Confirmações por Rede

Rede	Confirmações	Justificativa
Liquid (Depix)	2	Finalidade após 2 conf (~2 min). Documentação Blockstream.
BTC (baixo risco)	1	Valores < 0.01 BTC, liberação rápida
BTC (médio risco)	3	Valores 0.01–0.1 BTC
BTC (alto risco)	6	Valores > 0.1 BTC ou usuário novo

Tabela 1.2: Confirmações por Rede

2. Arquitetura do Sistema

2.1 Visão Geral dos Componentes

O sistema é estruturado com um Payout Service central que orquestra executores por rede. Cada executor implementa uma interface comum e se comunica com seu respectivo node via RPC.

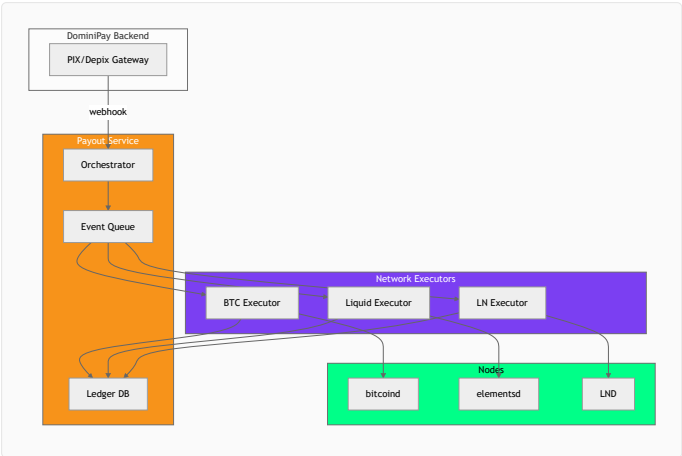


Figura 2.1: Diagrama de Arquitetura do Sistema

2.2 Interface dos Executores

```
type NetworkExecutor interface {
    // Validação
    ValidateAddress(addr string) (bool, error)
    ValidateInvoice(invoice string) (*InvoiceInfo, error) // LN only

    // Execução
    SendPayout(req PayoutRequest) (*PayoutResult, error)

    // Monitoramento
    WatchConfirmations(txid string) ←chan ConfirmationEvent
    GetBalance() (*Balance, error)
}
```

Listagem 2.1: Interface NetworkExecutor

2.3 Máquina de Estados

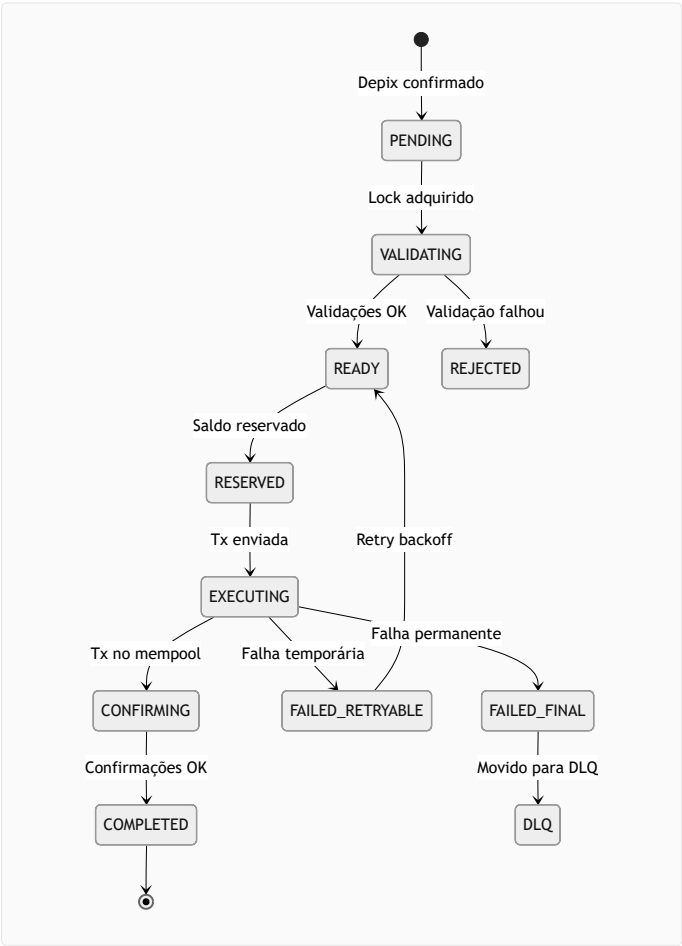


Figura 2.2: Máquina de Estados do Payout

De	Para	Trigger	Ação
PENDING	VALIDATING	Evento de fila	Adquirir lock por <code>payment_id</code>
READY	RESERVED	Validação OK	BTC/Liquid: Coin reservation (marcar UTXOs) LN: <code>reserved_amount += X</code>
RESERVED	EXECUTING	Saldo reservado	Criar/assinar/broadcast tx
CONFIRMING	COMPLETED	Confirmações atingidas	Liberar lock, persistir proof
FAILED_RETRYABLE	READY	Timer backoff	Incrementar <code>retry_count</code>
FAILED_RETRYABLE	FAILED_FINAL	<code>retry_count ≥ 3</code>	Mover para DLQ

Tabela 2.1: Transições de Estado

3. Policy Table v1

Valores iniciais para implementação, prontos para ajuste baseado em métricas de produção.

3.1 Lightning Network

3.1.1 Parâmetros por Faixa de Valor

Faixa	timeout	cltv_limit	fee_limit	MPP	max_parts	max_shard
< 100k sats	30s	40	5000 ppm, min 5, max 500 sat	OFF	6	50k sat
100k–1M sats	60s	60	3000 ppm, min 20, max 2k sat	ON	10	100k sat
> 1M sats	90s	80	2000 ppm, min 50, max 5k sat	ON	12	200k sat

Tabela 3.1: Parâmetros LN por Faixa

3.1.2 Regras Auxiliares LN

- Validar invoice: expiry , num_satoshis , payment_hash , destination
- Bloquear se now + timeout_seconds estiver a menos de 20% do expiry
- Zero-amount: aceitar somente se payout define valor; enviar via amt_msat
- Persistir payment_preimage + status após SUCCEEDED

3.2 Bitcoin On-Chain

Parâmetro	Valor
Confirmações (baixo risco)	1
Confirmações (médio risco)	3
Confirmações (alto risco)	6
RBF	ON por padrão
CPFP	Opcional (quando controlamos change)
Fee estimation targets	2 / 6 / 12 blocos
Fee fallback	10 sat/vB

Tabela 3.2: Parâmetros BTC

3.3 Liquid Network

Parâmetro	Valor
Depix confirmações	2
Payout confirmações	2
Fee fallback	0.1 sat/vB

Tabela 3.3: Parâmetros Liquid

3.4 Treasury Wallet Limits

Limites para as carteiras da tesouraria operacional (não-custodial de clientes):

Rede	min_balance	target_balance	max_balance
BTC	0.05 BTC	0.2 BTC	0.5 BTC
Liquid (L-BTC)	0.02 L-BTC	0.1 L-BTC	0.3 L-BTC
Lightning	500k sats	2M sats	5M sats

Tabela 3.4: Limites de Treasury Wallet

Importante: Reposição da cold → treasury wallet requer **aprovação humana** via dashboard ou CLI. Nunca automático.

4. Thresholds em BRL

Taxa de conversão de referência: 1 BTC = R\$ 600.000 (ajustar conforme mercado).

4.1 Faixas de Valor

Faixa	BRL	Sats	Classificação
Pequeno	< R\$ 600	< 100k	Baixo risco
Médio	R\$ 600 – R\$ 6.000	100k – 1M	Médio risco
Grande	> R\$ 6.000	> 1M	Alto risco

Tabela 4.1: Faixas de Valor em BRL

4.2 SLOs por Rede

4.2.1 Lightning Network

Métrica	Target	Janela
Taxa de sucesso	≥ 98%	24h rolling
Latência (p50)	≤ 15s	24h rolling
Latência (p99)	≤ 120s	24h rolling
Fee médio pago	≤ 0.3% do valor	Mensal

Tabela 4.2: SLOs Lightning

4.2.2 Bitcoin On-Chain

Métrica	Target	Janela
Taxa de broadcast	≥ 99.5%	24h
Tempo até 1 conf (p50)	≤ 15 min	24h
Tempo até 1 conf (p99)	≤ 60 min	24h
Tx stuck (> 24h)	≤ 0.1%	Semanal

Tabela 4.3: SLOs Bitcoin

4.3 Limites Operacionais

4.3.1 Por Payout

Rede	Mínimo	Máximo
Lightning	R\$ 1 (~170 sat)	R\$ 60.000 (~10M sat)
BTC	R\$ 60 (~10k sat)	R\$ 300.000 (~50M sat)
Liquid	R\$ 6 (~1k sat)	R\$ 300.000 (~50M sat)

Tabela 4.4: Limites por Payout

4.3.2 Por Usuário (Diário)

Tier	Limite Diário
Básico	R\$ 10.000
Verificado	R\$ 50.000
Premium	R\$ 200.000

Tabela 4.5: Limites por Usuário

5. Especificação de Validações

5.1 Códigos de Erro

5.1.1 Validação (4xx)

Código	HTTP	Descrição	Retryable
PAYOUT_INVALID_ADDRESS	400	Endereço inválido	✗
PAYOUT_MIXED_CASE_BECH32	400	Bech32 com mixed-case	✗
PAYOUT_INVALID_INVOICE	400	Invoice LN malformada	✗
PAYOUT_INVOICE_EXPIRED	400	Invoice já expirada	✗
PAYOUT_INVOICE_NEAR_EXPIRY	400	Invoice < 20% tempo restante	✗
PAYOUT_AMOUNT_MISMATCH	400	Valor difere do esperado	✗
PAYOUT_DUPLICATE	409	payment_id já processado	✗
PAYOUT_LIMIT_EXCEEDED	429	Limite diário excedido	✗

Tabela 5.1: Códigos de Erro de Validação

5.1.2 Execução (5xx)

Código	HTTP	Descrição	Retryable
PAYOUT_INSUFFICIENT_BALANCE	503	Saldo insuficiente	✓
PAYOUT_NETWORK_UNAVAILABLE	503	Rede indisponível	✓
PAYOUT_CIRCUIT_OPEN	503	Circuit breaker aberto	✓
PAYOUT_LN_NO_ROUTE	502	Sem rota LN disponível	✓
PAYOUT_LN_FEE_TOO_HIGH	502	Fee LN excede limite	✓
PAYOUT_TX_REJECTED	502	Tx rejeitada pelo node	✗

Tabela 5.2: Códigos de Erro de Execução

5.2 Validações por Rede

5.2.1 BTC Address

```
func ValidateBTCAddress(addr string) error {
    // 1. Verificar mixed-case (BIP-173)
    if hasMixedCase(addr) && strings.HasPrefix(strings.ToLower(addr), "bc1") {
        return ErrMixedCaseBech32
    }
    // 2. Chamar bitcoind validateaddress
    resp, _ := rpc.Call("validateaddress", addr)
    if !resp.IsValid {
        return ErrInvalidAddress
    }
    return nil
}
```

Listagem 5.1: Validação de Endereço BTC

5.2.2 Lightning Invoice

```
func ValidateLNInvoice(invoice string, expectedAmount int64) error {
    decoded, err := lnd.DecodePayReq(invoice)
    if err != nil {
        return ErrInvalidInvoice
    }

    // Expiração
    expiresAt := decoded.Timestamp + decoded.Expiry
    if time.Now().Unix() > expiresAt {
        return ErrInvoiceExpired
    }

    // 80% do expiry
    remaining := expiresAt - time.Now().Unix()
    if remaining < int64(decoded.Expiry)*0.2 {
        return ErrInvoiceNearExpiry
    }

    // Amount
    if decoded.NumSatoshis == 0 {
        // Zero-amount: aceitar se expectedAmount > 0
    } else if decoded.NumSatoshis != expectedAmount {
        return ErrAmountMismatch
    }

    return nil
}
```

Listagem 5.2: Validação de Invoice LN

5.3 Regras de Negócio

```
-- Constraint de unicidade
CREATE UNIQUE INDEX idx_payout_idempotency
ON payouts (payment_id, network);

-- Lock + Check
BEGIN;
SELECT * FROM payouts
WHERE payment_id = $1 AND network = $2
FOR UPDATE NOWAIT;

INSERT INTO payouts (payment_id, network, status, ...)
VALUES ($1, $2, 'PENDING', ...)
```



```
ON CONFLICT DO NOTHING
RETURNING id;
COMMIT;
```

Listagem 5.3: Idempotência via SQL

6. Fluxos de Execução

6.1 Bitcoin Flow

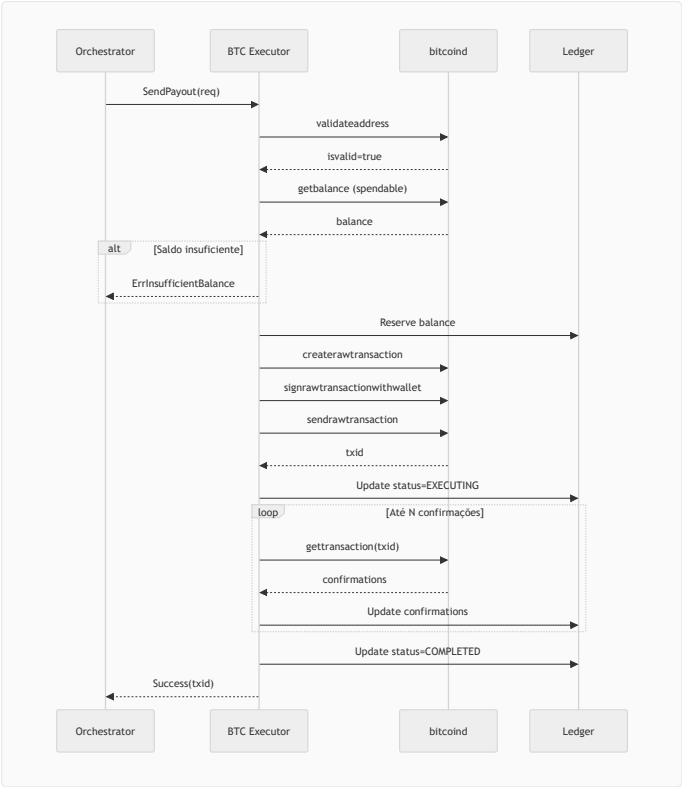


Figura 6.1: Fluxo de Payout BTC

6.2 Lightning Flow

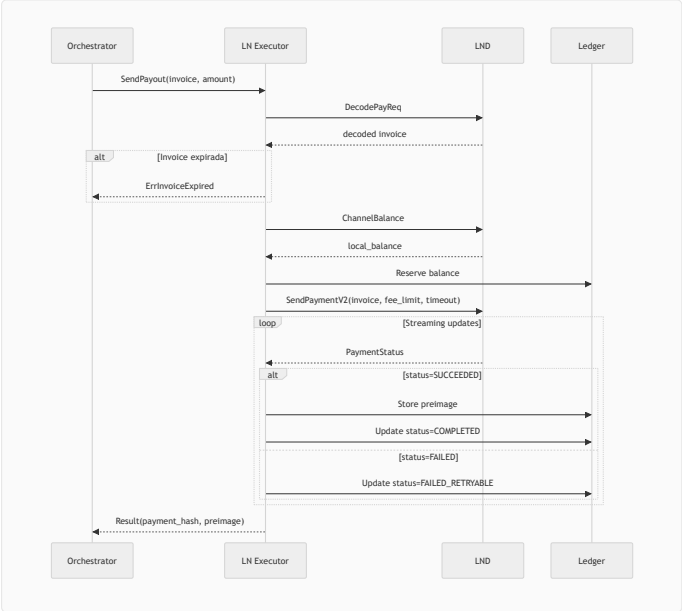


Figura 6.2: Fluxo de Payout Lightning

7. Resiliência

7.1 Circuit Breaker

Circuit breakers são **independentes por rede**. BTC aberto NÃO bloqueia LN ou Liquid.

```
type CircuitBreaker struct {
    State      string    // CLOSED, OPEN, HALF_OPEN
    FailureCount int       //
    FailureThreshold int    // default: 5
    OpenDuration time.Duration // default: 60s
    LastFailure time.Time
    Queue      []PayoutRequest // Fila preservada
    MaxQueueSize int        // default: 100 (backlog limit)
}

func (cb *CircuitBreaker) Execute(fn func() error) error {
    if cb.State == "OPEN" {
        if time.Since(cb.LastFailure) > cb.OpenDuration {
            cb.State = "HALF_OPEN"
        } else {
            // Shed load se backlog exceder limite
            if len(cb.Queue) >= cb.MaxQueueSize {
                return ErrBackLogFull // Retryable, não enfileira
            }
            return ErrCircuitOpen // Enqueue
        }
    }

    err := fn()
    if err != nil {
        cb.FailureCount++
        cb.LastFailure = time.Now()
        if cb.FailureCount >= cb.FailureThreshold {
            cb.State = "OPEN"
        }
        return err
    }

    cb.FailureCount = 0
    cb.State = "CLOSED"
    return nil
}
```

Listagem 71: Circuit Breaker com Backlog Limit

Proteção Financeira: Quando circuit abre, payouts são **enfileirados até o limite**. Se `len(queue) >= MaxQueueSize`, novos payouts são rejeitados com `ErrBackLogFull` (retryable). Isso evita "fila de dívida" em burst de Depix.

Parâmetro	Valor Padrão	Descrição
MaxQueueSize	100 por rede	Limite de backlog; acima disso, shed load
FailureThreshold	5	Falhas consecutivas para abrir circuit
OpenDuration	60s	Tempo em OPEN antes de tentar HALF_OPEN

Tabela 70: Parâmetros do Circuit Breaker

7.2 Retry Policy

7.2.1 Lightning Network

Tentativa	Delay	Ajuste
1	imediato	Parâmetros normais
2	5s	<code>max_hops</code> reduzido
3	15s	<code>fee_limit</code> +20%
4	—	Marcar FAILED_RETRYABLE

Tabela 71: Retry Policy LN

7.2.2 Condições de Abort

Condição	Ação
Invoice > 80% expiry consumido	Abortar, solicitar nova invoice
<code>insufficient_balance</code>	Abortar imediatamente, alertar ops
<code>payment_hash_reuse</code>	Não retry, marcar duplicate

Tabela 72: Condições de Abort

8. Auditoria e Observabilidade

8.1 Schema de Auditoria

```
CREATE TABLE payout_audit (  
  id BIGSERIAL PRIMARY KEY,  
  payout_id BIGINT REFERENCES payouts(id),  
  payment_id VARCHAR(64) NOT NULL,  
  network VARCHAR(10) NOT NULL,  
  
  -- Prova  
  txid VARCHAR(64),      -- BTC/Liquid  
  payment_hash VARCHAR(64), -- LN  
  preimage_hash VARCHAR(64), -- SHA256(preimage)  
  
  -- Hash chain  
  prev_hash VARCHAR(64),  
  record_hash VARCHAR(64), -- SHA256(prev || payment_id || txid)  
  
  -- Metadata  
  amount_sats BIGINT,  
  fee_paid_sats BIGINT,  
  confirmations INT,  
  status VARCHAR(20),  
  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);  
  
-- Trigger para impedir UPDATE (WORM)  
CREATE TRIGGER audit_immutable  
BEFORE UPDATE ON payout_audit  
FOR EACH ROW EXECUTE FUNCTION prevent_audit_update();
```

Listagem 8.1: Schema de Auditoria Append-Only

8.2 Métricas Prometheus

```
# Counters  
domini_payout_total{network, status}  
domini_payout_retries_total{network}  
  
# Histograms  
domini_payout_duration_seconds{network}  
domini_payout_confirmations_wait_seconds{network}  
domini_ln_fee_paid_sats{bucket}  
  
# Gauges  
domini_hot_wallet_balance_sats{network}  
domini_utxo_count{network}  
domini_circuit_breaker_state{network} # 0=closed, 1=open, 2=half_open
```

Listagem 8.2: Métricas Prometheus

8.3 Alertas Recomendados

Alerta	Condição	Severidade
HotWalletLow	balance < min_balance por 5 min	Warning
HotWalletCritical	balance < min_balance * 0.5	Critical
CircuitBreakerOpen	state == OPEN por 2 min	Critical
HighFailureRate	failed / total > 0.1 em 5 min	Warning
UTXOFragmentation	utxo_count > 100	Warning
PayoutLatencyHigh	p99 > 120s (LN) ou > 30min (BTC)	Warning

Tabela 8.1: Alertas Recomendados

9. Runbook Operacional

9.1 RBF - Replace-By-Fee

Para transações BTC stuck no mempool por mais de 1 hora:

```
# 1. Verificar status da tx
bitcoin-cli gettransaction <txid>

# 2. Calcular novo fee (target: próximo bloco)
NEW_FEE=$(bitcoin-cli estimatesmartfee 2 | jq '.feerate')

# 3. Bump fee
bitcoin-cli bumpfee <txid> '{"fee_rate": '$NEW_FEE'}'

# 4. Verificar nova tx
bitcoin-cli gettransaction <new_txid>
```

Listagem 9.1: Procedimento RBF

Pré-requisito: Tx original deve ter sido criada com `replaceable=true` .

9.2 DLQ - Dead Letter Queue

9.2.1 O que vai para DLQ

- Payouts com 3+ falhas
- Erros não-retryable
- Timeouts esgotados

9.2.2 Processamento

```
# Listar itens
curl https://api.domini.pay/v1/admin/dlq?limit=50

# Retry forçado
curl -X POST https://api.domini.pay/v1/admin/dlq/<id>/retry

# Resolução manual
curl -X POST https://api.domini.pay/v1/admin/dlq/<id>/resolve \
-d '{"resolution": "manual_refund", "notes": "PIX"}'
```

Listagem 9.2: Comandos DLQ

9.3 Consolidação de UTXOs

Trigger	Ação
UTXOs > 50	Consolidar no domingo 02:00 UTC
Fees < 5 sat/vB	Consolidar imediatamente

Tabela 9.1: Triggers de Consolidação

```
# Contar UTXOs
bitcoin-cli listunspent | jq 'length'

# Consolidar
bitcoin-cli sendtoaddress "<hot_wallet>" \
$(bitcoin-cli getbalance) \
"" "" true true null "unset" null $FEE_RATE
```

Listagem 9.3: Consolidação Manual

10. Checklist de Production Readiness

10.1 Funcionalidades Core

- ☐ BTC Executor: send, watch, confirm
- ☐ Liquid Executor: send, watch, confirm
- ☐ LN Executor: decode, send, track
- ☐ State machine completa
- ☐ Idempotência por payment_id + network
- ☐ Lock distribuído (DB ou Redis)
- ☐ Reserva de saldo pré-envio

10.2 Resiliência

- ☐ Circuit breaker por rede (com fila)
- ☐ Retry policy com backoff
- ☐ DLQ para falhas finais
- ☐ Fallback de fee estimation
- ☐ Timeout configurável por rede

10.3 Segurança

- ☐ Preimage LN criptografado em repouso
- ☐ Hot wallet limits enforced
- ☐ Aprovação humana para reposição
- ☐ Auditoria append-only
- ☐ Hash chain para integridade

10.4 Operacional

- ☐ Consolidação de UTXOs (cron)
- ☐ Métricas Prometheus
- ☐ Alertas configurados
- ☐ Runbook de DLQ
- ☐ Dashboard de monitoramento

10.5 Testes

- ☐ Unit tests: 80%+ cobertura
- ☐ Integration tests: cada executor
- ☐ Chaos tests: kill mid-payout
- ☐ Replay test: idempotência validada
- ☐ Load test: throughput esperado

10.6 SLOs Propostos

Métrica	Target	Medição
Disponibilidade	99.9%	Uptime do Payout Service
Latência LN (p99)	< 120s	Da requisição ao COMPLETED
Latência BTC (p99)	< 60 min	Da requisição à 1ª confirmação
Taxa de sucesso	> 98%	Payouts COMPLETED / Total
Falhas não recuperadas	< 0.1%	Payouts em DLQ / Total

Tabela 10.1: SLOs Propostos

11. Configuração de Referência

```
ln:
  timeout_seconds:
    small: 30 # < 100k sats
    medium: 60 # 100k-1M sats
    large: 90 # > 1M sats
  cltv_limit:
    small: 40
    medium: 60
    large: 80
  fee_limit_ppm:
    small: 5000 # 0.5%
    medium: 3000 # 0.3%
    large: 2000 # 0.2%
  fee_limit_min_sat:
    small: 5
    medium: 20
    large: 50
  fee_limit_max_sat:
    small: 500
    medium: 2000
    large: 5000
  mpp:
    enable_threshold_sat: 100000
    max_parts:
      small: 6
      medium: 10
      large: 12
    max_shard_size_sat:
      small: 50000
      medium: 100000
      large: 200000
  retry:
    max_attempts: 3
    backoff_seconds: [5, 15, 45]
    expiry_threshold_pct: 0.8

btc:
  confs_final:
    low: 1
    medium: 3
    high: 6
  rbf: true
  fee_targets_blocks: [2, 6, 12]
  fee_fallback_sat_vb: 10

liquid:
  depix_confs: 2
  payout_confs: 2
  fee_fallback_sat_vb: 0.1

hot_wallet:
  btc:
    min_balance_sat: 5000000 # 0.05 BTC
    target_balance_sat: 20000000 # 0.2 BTC
    max_balance_sat: 50000000 # 0.5 BTC
  liquid:
    min_balance_sat: 2000000
    target_balance_sat: 10000000
    max_balance_sat: 30000000
  lightning:
    min_balance_sat: 500000
    target_balance_sat: 2000000
    max_balance_sat: 5000000

circuit_breaker:
  failure_threshold: 5
  open_duration_seconds: 60

consolidation:
  utxo_threshold: 50
  schedule: "0 2 * * 0" # Domingo 02:00
```

Listagem 11.1: Configuração YAML Completa

Documento aprovado para implementação. Revisar semanalmente a taxa de conversão BRL e mensalmente as métricas de SLO para ajustes na Policy Table.